Name : S.HEMAA
Register no: 250701238
Experiment Name : Experiment to understand pandas Library use cases in data Science

In [1]:
```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
```

In [2]:
```
df=pd.DataFrame(list)
df
```

Out[2]:

	0	1	2
0	1	Smith	50000
1	2	Jones	60000

In [3]:
```
df.columns=['Empd','Name','Salary']
df
```

Out[3]:

	Empd	Name	Salary
0	1	Smith	50000
1	2	Jones	60000

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   Empd    2 non-null      int64 
 1   Name    2 non-null      str   
 2   Salary  2 non-null      int64 
dtypes: int64(2), str(1)
memory usage: 180.0 bytes
```

In [7]:
```
df=pd.read_csv("50_Startups.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     str    
 4   Profit           50 non-null     float64
dtypes: float64(4), str(1)
memory usage: 2.1 KB
```

In [8]:
```
df.head()
```

Out[8]:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

In [9]:
```
df.tail()
```

Out[9]:

	R&D Spend	Administration	Marketing Spend	State	Profit
45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	45173.06	California	14681.40

In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv("employee.csv")
df.head()
```

Out[2]:

	emp_id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SLAKASHI U G	8000

In [3]:
```
df.tail()
```

Out[3]:

	emp_id	name	salary
15	16	SANJAY D	9200
16	17	KARTHIK N	7800
17	18	PAVITHRA L	6400
18	19	ROHIT M	8800
19	20	AKSHAYA B	7100

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp_id  20 non-null     int64
 1   name    20 non-null     str
 2   salary  20 non-null     int64
dtypes: int64(2), str(1)
memory usage: 612.0 bytes
```

In [5]:
```
df.salary
```

Out[5]:
```
0      5000
1      6000
2      7000
3      5000
4      8000
5      9000
6      6500
7      7500
8      5500
9     10000
10     6200
11     8500
12     7200
13     6800
14     5800
15     9200
16     7800
17     6400
18     8800
19     7100
Name: salary, dtype: int64
```

In [6]:
```
type(df.salary)
```

Out[6]:
```
pandas.Series
```

In [7]:
```
df.salary.mean()
```

Out[7]:
```
np.float64(7165.0)
```

In [8]:
```
df.salary.median()
```

Out[8]:
```
np.float64(7050.0)
```

In [9]:
```
df.salary.mode()
```

Out[9]:
```
0    5000
Name: salary, dtype: int64
```

In [10]:
```
df.salary.var()
```

Out[10]: np.float64(2037131.5789473683)

In [11]:
```
df.salary.std()
```

Out[11]: np.float64(1427.2811842616607)

In [12]:
```
df.describe()
```

Out[12]:

	emp_id	salary
count	20.00000	20.000000
mean	10.50000	7165.000000
std	5.91608	1427.281184
min	1.00000	5000.000000
25%	5.75000	6150.000000
50%	10.50000	7050.000000
75%	15.25000	8125.000000
max	20.00000	10000.000000

In [13]:
```
df.describe(include='all')
```

Out[13]:

	emp_id	name	salary
count	20.00000	20	20.000000
unique	NaN	20	NaN
top	NaN	SREE VARSSINI K S	NaN
freq	NaN	1	NaN
mean	10.50000	NaN	7165.000000
std	5.91608	NaN	1427.281184
min	1.00000	NaN	5000.000000
25%	5.75000	NaN	6150.000000
50%	10.50000	NaN	7050.000000
75%	15.25000	NaN	8125.000000
max	20.00000	NaN	10000.000000

In [14]:
```
empCol=df.columns
empCol
```

Out[14]: Index(['emp_id', 'name', 'salary'], dtype='str')

In [15]:
```
emparray=df.values
emparray
```

Out[15]:
```
array([[1, 'SREE VARSSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
       [5, 'SLAKASHI U G', 8000],
       [6, 'ARUN KUMAR', 9000],
       [7, 'PRIYA R', 6500],
       [8, 'KAVIN S', 7500],
       [9, 'DIVYA M', 5500],
       [10, 'RAHUL P', 10000],
       [11, 'NITHYA K', 6200],
       [12, 'VIGNESH R', 8500],
       [13, 'ANJALI S', 7200],
       [14, 'HARISH V', 6800],
       [15, 'MEENA T', 5800],
       [16, 'SANJAY D', 9200],
       [17, 'KARTHIK N', 7800],
       [18, 'PAVITHRA L', 6400],
       [19, 'ROHIT M', 8800],
       [20, 'AKSHAYA B', 7100]], dtype=object)
```

In [16]:
```
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF
```

Out[16]:

	emp_id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SLAKASHI U G	8000
5	6	ARUN KUMAR	9000
6	7	PRIYA R	6500
7	8	KAVIN S	7500
8	9	DIVYA M	5500
9	10	RAHUL P	10000
10	11	NITHYA K	6200
11	12	VIGNESH R	8500
12	13	ANJALI S	7200
13	14	HARISH V	6800
14	15	MEENA T	5800
15	16	SANJAY D	9200
16	17	KARTHIK N	7800
17	18	PAVITHRA L	6400
18	19	ROHIT M	8800
19	20	AKSHAYA B	7100

In [ ]: